MODULE *streams*

The stream layer of the version 2 *Zcash P2P* protocol (draft *ZIP*, zcash/zips#1344, "Stream Layer"), as realised by *QUIC*.

A connection carries many streams. Each stream is reliable and ordered on its own, but streams are mutually independent: data on one stream is never delayed by another (draft: "Transport Requirements"). The model therefore keeps one in-flight queue PER STREAM rather than one inbox per connection as the legacy model does; delivery picks any stream, which yields every cross-stream interleaving the transport allows.

Two control signals can overtake queued data in *QUIC* and are modeled as flags rather than queue elements: *RESET_STREAM* (*in_reset*, *sent by the peer on its own sending direction*)*and STOP_SENDING*(*stop*, *the peer asking us to stop sending*).*FIN is ordered after the data it follows and is a queue sentinel.*

This module defines data shapes and pure helpers only; *the actions live in protocol.tla.*

EXTENDS *Naturals*, *Sequences*, *FiniteSets*

CONSTANT *MaxStreams* *stream slots per opener per connection*

Stream types(*draft* : "Stream Types").*The type is the first byte written on every stream*; *the receiver learns it only by consuming that byte.*

$HandshakeType \triangleq$ "0x00" *bidirectional*, *initiator only*, *exactly one*
$GetHeadersType \triangleq$ "0x01" *bidirectional request stream*
$GetBlocksType \triangleq$ "0x02" *bidirectional request stream*
$BlockAnnType \triangleq$ "0x10" *unidirectional*, *long − lived*
$RequestTypes \triangleq \{GetHeadersType,\ GetBlocksType\}$
$AnnTypes \triangleq \{BlockAnnType\}$
$StreamTypes \triangleq \{HandshakeType\} \cup RequestTypes \cup AnnTypes$
$BidiTypes \triangleq \{HandshakeType\} \cup RequestTypes$

Application error codes(*draft* : "Application Error Codes").

$ErrorCodes \triangleq \{$"NO_ERROR", "PROTOCOL_ERROR", "UNSUPPORTED_STREAM_TYPE", "OBSOLETE", "SELF_CONNECTION", "FLOOD", "MISBEHAVIOR", "CANCELLED", "REFUSED", "INTERNAL_ERROR"$\}$

$NoCode \triangleq$ "none"

Every queue element is a record with a kind field so that TLC can compare them : *the type byte that opens a stream*, *the FIN sentinel of a finished sending direction*, *and the protocol records of records.tla.*

$TypeByte(t) \triangleq [kind \mapsto$ "type", $t \mapsto t]$
$FIN \triangleq [kind \mapsto$ "fin"$]$

Stream identifiers for the connection between n and m : *who opened it*, *and a slot number.Slots are reused once both peers have closed the stream.*

$StreamIds(n,\ m) \triangleq \{n,\ m\} \times (1\ ..\ MaxStreams)$

A peer s local view of one stream. Queued data flows TOWARD the owner of the record: a sender appends to the remote peer's record, exactly as the legacy model appends to the remote inbox.

status "none" (slot free) | "open" | "closed" (done locally, peer may not be)
rtype stream type once known; "unknown" until the type byte is consumed
inq data in flight toward this peer, *FIN*-terminated when finished
in_done the peer's sending direction has been consumed to its *FIN*
(TRUE from the start on streams with no incoming direction)
in_reset error code of a *RESET_STREAM* from the peer, observable any time
out this peer's own sending direction: "open" | "finished" | "reset" | "na"
stop error code of a *STOP_SENDING* from the peer for our direction

$$\begin{aligned} \mathit{NullStream} \triangleq [&\mathit{status} \mapsto \text{“none”},\\ &\mathit{rtype} \mapsto \text{“unknown”},\\ &\mathit{inq} \mapsto \langle\rangle,\\ &\mathit{in\_done} \mapsto \text{TRUE},\\ &\mathit{in\_reset} \mapsto \mathit{NoCode},\\ &\mathit{out} \mapsto \text{“na”},\\ &\mathit{stop} \mapsto \mathit{NoCode}] \end{aligned}$$

A locally closed stream collapses to one canonical record so that the state space does not retain the history of how it was closed.

$$\mathit{ClosedStream} \triangleq [\mathit{NullStream} \text{ EXCEPT } !.\mathit{status} = \text{“closed”}]$$

The opener's record of a stream it just opened with type t.

$$\begin{aligned} \mathit{OpenerStream}(t) \triangleq [&\mathit{status} \mapsto \text{“open”},\\ &\mathit{rtype} \mapsto t,\\ &\mathit{inq} \mapsto \langle\rangle,\\ &\mathit{in\_done} \mapsto t \notin \mathit{BidiTypes},\\ &\mathit{in\_reset} \mapsto \mathit{NoCode},\\ &\mathit{out} \mapsto \text{“open”},\\ &\mathit{stop} \mapsto \mathit{NoCode}] \end{aligned}$$

The requester's record of a request stream: the request has been written and the sending direction finished in the same step (draft: "Request Streams", step 1).

$$\mathit{RequesterStream}(t) \triangleq [\mathit{OpenerStream}(t) \text{ EXCEPT } !.\mathit{out} = \text{“finished”}]$$

The remote peer's record of the same stream: the type byte is the first queued element, followed by any records written at open time.

$$\begin{aligned} \mathit{PeerStream}(t,\ \mathit{payload}) \triangleq [&\mathit{status} \mapsto \text{“open”},\\ &\mathit{rtype} \mapsto \text{“unknown”},\\ &\mathit{inq} \mapsto \langle \mathit{TypeByte}(t)\rangle \circ \mathit{payload},\\ &\mathit{in\_done} \mapsto \text{FALSE}, \end{aligned}$$

$in\_reset \mapsto NoCode,$
$out \mapsto \text{IF } t \in BidiTypes \text{ THEN } \text{“open”} \text{ ELSE } \text{“na”},$
$stop \mapsto NoCode]$

Transport-side delivery helpers. Writes to a stream the receiver has already closed are dropped, as *QUIC* discards data after *STOP_SENDING*.

$PutData(s,\, x) \triangleq \text{IF } s.status = \text{“open”} \text{ THEN } [s \text{ EXCEPT } !.inq = Append(@,\, x)] \text{ ELSE } s$
$PutReset(s,\, code) \triangleq \text{IF } s.status = \text{“open”} \text{ THEN } [s \text{ EXCEPT } !.in\_reset = code] \text{ ELSE } s$
$PutStop(s,\, code) \triangleq \text{IF } s.status = \text{“open”} \text{ THEN } [s \text{ EXCEPT } !.stop = code] \text{ ELSE } s$

Refusing a stream of type t (draft: “Stream Types”): cancel the peer's sending direction and, if the stream is bidirectional, also reset our own sending direction with the same code. Both signals land on the peer's record and may be observed in either order.

$Refuse(s,\, t,\, code) \triangleq \text{IF } t \in BidiTypes \text{ THEN } PutReset(PutStop(s,\, code),\, code) \text{ ELSE } PutStop(s,\, code)$

A stream is done for its owner once both directions are over; it then collapses to *ClosedStream*.

$Done(s) \triangleq s.in\_done \land s.out \in \{\text{“finished”},\, \text{“reset”},\, \text{“na”}\}$
$Collapse(s) \triangleq \text{IF } Done(s) \text{ THEN } ClosedStream \text{ ELSE } s$

Head-of-queue classification.

$IsTypeByte(x) \triangleq x.kind = \text{“type”}$
$IsFin(x) \triangleq x.kind = \text{“fin”}$